KV Cache Engine
Software Overview

LonghornSilicon — Compiler Co-Design Orientation
Block 2 of 4 — KV Cache Compression / Decompression

Version: 0.2 (ChannelQuant codec)
Date: 2026-07-03
Audience: Compiler team, runtime engineers

Contents

1 Purpose **3**

2 What the Block Does **3**
2.1 Compression Pipeline 3

3 Reference Models **3**
3.1 Directory Layout 3
3.2 Which Models Are Available 4
3.3 Build Everything 4

4 What the Compiler Should Emit **4**
4.1 Worked Example: Attention with KV Cache 4

5 Integration Phases **5**

6 Interaction with Block 1 (Precision Controller) **5**

7 Open Questions for the Compiler Team **5**

1. Purpose

This document orients the compiler team on the KV cache engine block. It covers: what the block does, how to use the reference models, what the compiler should emit, and what integration phases look like.

For the formal interface contract, see the ISA specification (`docs/isa/kv_cache_engine_isa.pdf`). For the API reference, see `docs/reference_model_api.pdf`.

2. What the Block Does

The KV cache engine sits between the attention compute unit (block 1) and the memory hierarchy controller (block 4). It compresses key and value vectors before writing to on-chip SRAM, and decompresses them when the attention unit needs them for dot-product computation.

- **On write**: incoming FP16 K/V vectors are compressed via **ChannelQuant** (per-channel INT4 keys grouped over G tokens + optional FP16 outlier channels; per-token INT4 values) and stored in SRAM.
- **On read**: compressed entries are decompressed back to **fp32** coordinates and streamed to the attention unit via AXI-Stream.
- **Eviction**: when SRAM is full, the block signals the memory hierarchy controller to spill entries to DRAM.

2.1 Compression Pipeline

1. **Key group buffer** — accumulate G key tokens (`residual_buffer`).
2. **Per-channel scale** — per-channel max over the group → D frozen FP16 scales (`amax_unit` + `scale_bank`); values use a per-token FP16 scale.
3. **Quantize** — keep channels → INT4; the top-k outlier channels (CQ-4+) are held FP16 from a static ROM mask. Values → INT4 (INT8 for the CQ-8 tier).
4. **Store** — a unified per-channel SRAM record (keep → {scale, INT4}; outlier → {fp16, code +1}).

Decompression is per channel: $\hat{x}_c = \text{code}_c \cdot \text{scale}_c$ in fp32; outlier channels replay their stored FP16 directly. No rotation, no Johnson–Lindenstrauss, no Lloyd–Max codebook (those were the retired TurboQuant+ codec).

3. Reference Models

3.1 Directory Layout

```
sw/
+-- README.md                       <- orientation (this doc's markdown form)
+-- reference_model/
    +-- README.md                   <- API reference + build instructions
    +-- Makefile
```

```
    +-- channelquant_ref.hpp/.cpp <- ChannelQuant C++ reference (shipped
        codec)
    +-- test_channelquant_ref.cpp <- 9 golden-vector tests
    +-- kv_cache_engine_ref.*     <- retired TurboQuant+ reference (archival
         only)
# Python algorithm reference + golden vectors: ../channelquant/reference/
```

3.2 Which Models Are Available

Codec	Status	C++ API	Python
ChannelQuant (shipped)	Bit-exact vs RTL + golden	`lhsi::cq::compress_*`	`../channelquant` ref
TurboQuant+ (retired)	archival regression only	`lhsi::KVCacheEngine`	—

3.3 Build Everything

```
cd sw/reference_model
make test-cq      # ChannelQuant C++ tests vs the 9 golden vectors
make test-all     # legacy TQ + ChannelQuant + Python tests
make shared       # libkv_cache_engine_ref.so
make static       # libkv_cache_engine_ref.a
```

Requires Python 3.10+, NumPy, and a C++17 compiler.

4. What the Compiler Should Emit

A compiler backend targeting the KV cache engine emits sequences of the operations defined in the ISA spec (§4 of `docs/isa/kv_cache_engine_isa.pdf`):

Operation	C API	What it does
`KV_QUERY`	`lhsi_kv_info()`	Read hardware config
`KV_RESET`	`lhsi_kv_reset()`	Clear SRAM, reset FSM
`KV_COMPRESS_KEY`	`lhsi_kv_compress_key()`	Compress + store key
`KV_COMPRESS_VALUE`	`lhsi_kv_compress_value()`	Compress + store value
`KV_DECOMPRESS_KEY`	`lhsi_kv_decompress_key()`	Read + decompress key
`KV_DECOMPRESS_VALUE`	`lhsi_kv_decompress_value()`	Read + decompress value

4.1 Worked Example: Attention with KV Cache

The shipped reference is head-level ChannelQuant (frozen in `../channelquant`): compress a whole KV head $[T, D]$, then reconstruct for attention.

```
import channelquant_ref as cq   # ../channelquant/reference

# Store phase: per-channel INT4 keys (grouped G), per-token INT4 values.
K_hat = cq.fq_per_channel(K, bits=4, G=128)          # or
    fq_per_channel_outlier for CQ-4+
V_hat = cq.fq_per_token(V, bits=4)
```

```
# Attention phase: use the reconstructed K_hat / V_hat.
scores = query @ K_hat.transpose(-1, -2) / sqrt(D)    # Q * K^T
# Route each S*V tile through the precision controller (block 1); see below.
```

5. Integration Phases

Phase	Timeline	Compiler targets	We provide
0	now	Python + C++ reference models	Models + ISA + tests
1	when FPGA arrives	AXI-Lite + AXI-Stream on Zynq	Vivado bitstream + driver
2	all 4 blocks done	Multi-block FPGA project	Integrated attention pipeline
3	post-tape-out	PCIe-attached chip	TSMC 16FFC silicon + driver

The ISA contract (`docs/isa/kv_cache_engine_isa.pdf`) is stable across all four phases. Only the runtime implementation changes — Python function call, AXI register write, or PCIe MMIO.

6. Interaction with Block 1 (Precision Controller)

The KV cache engine and precision controller work together during attention inference:

1. KV cache engine decompresses K/V vectors from SRAM
2. The attention unit computes $Q \cdot K^T$ scores
3. The precision controller decides INT8 vs FP16 per tile
4. The MAC array computes $\text{softmax}(S) \cdot V$ at the chosen precision

A compiler backend that targets both blocks emits interleaved `KV_DECOMPRESS_*` and `PC_PUSH_TILE` operations. The reference models for both blocks can be composed in the same process:

```
import channelquant_ref as cq                          # block 2 (KVCE)
from precision_controller_ref import PrecisionController  # block 1 (ACU)

pc = PrecisionController()

K_hat = cq.fq_per_channel(K, bits=4, G=128)            # decompressed keys
scores = query @ K_hat.transpose(-1, -2)               # Q * K^T
decision = pc.process_tile(scores)                     # True = FP16, False =
    INT8
```

7. Open Questions for the Compiler Team

1. **Granularity**: does the compiler emit individual compress/decompress ops, or a fused `kv_attention(Q, K_cache, V_cache)` op?

2. **Batch interface**: should the reference model support batched compress/decompress (multiple vectors in one call) for throughput?
3. **Async model**: should `KV_COMPRESS_KEY` be non-blocking with a separate completion query, matching the hardware pipeline?
4. **Cost model**: are placeholder cycle counts sufficient, or do you need post-synthesis numbers for the scheduler?